In [1]:

```python
# ----------------------------
# Project 2: Logistics Pricing & Route Optimization
# Python Notebook Workflow
# ----------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from ortools.constraint_solver import pywrapcp, routing_enums_pb2

# ----------------------------
# Step 1: Load Excel Data
# ----------------------------
df = pd.read_excel("logistics_data.xlsx")
print("Dataset Loaded:")
display(df.head())

# ----------------------------
# Step 2: Basic Cleaning
# ----------------------------
# Fill missing values if any
df.fillna(0, inplace=True)

# ----------------------------
# Step 3: Dynamic Pricing Simulation
# ----------------------------
# Base price per delivery
df['Base_Price'] = 100

# Price +20% if Booking_Demand > 10
df['Price'] = df['Booking_Demand'].apply(lambda x: x*10 if x>10 else 100)
display(df.head())

# ----------------------------
# Step 4: Route Optimization
# ----------------------------
cities = df['City'].unique()
num_cities = len(cities)

# Create random distance matrix between cities
```

```python
np.random.seed(42)
distance_matrix = np.random.randint(5,50,(num_cities,num_cities))
for i in range(num_cities):
    distance_matrix[i,i] = 0

# OR-Tools setup
manager = pywrapcp.RoutingIndexManager(num_cities, 1, 0)
routing = pywrapcp.RoutingModel(manager)

def distance_callback(from_index, to_index):
    return distance_matrix[manager.IndexToNode(from_index)][manager.IndexToNode(to_index)]

transit_callback_index = routing.RegisterTransitCallback(distance_callback)
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)

# Solve
search_params = pywrapcp.DefaultRoutingSearchParameters()
search_params.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
solution = routing.SolveWithParameters(search_params)

# Extract optimized route
route = []
if solution:
    index = routing.Start(0)
    while not routing.IsEnd(index):
        route.append(cities[manager.IndexToNode(index)])
        index = solution.Value(routing.NextVar(index))
    route.append(cities[manager.IndexToNode(index)])

print("Optimized Route:", route)

# ----------------------------
# Step 5: Visualization
# ----------------------------
# Plot Booking Demand vs Price
plt.figure(figsize=(10,5))
plt.scatter(df['Booking_Demand'], df['Price'], color='green')
plt.xlabel("Booking Demand")
plt.ylabel("Price")
plt.title("Dynamic Pricing Simulation")
plt.grid(True)
plt.show()
```

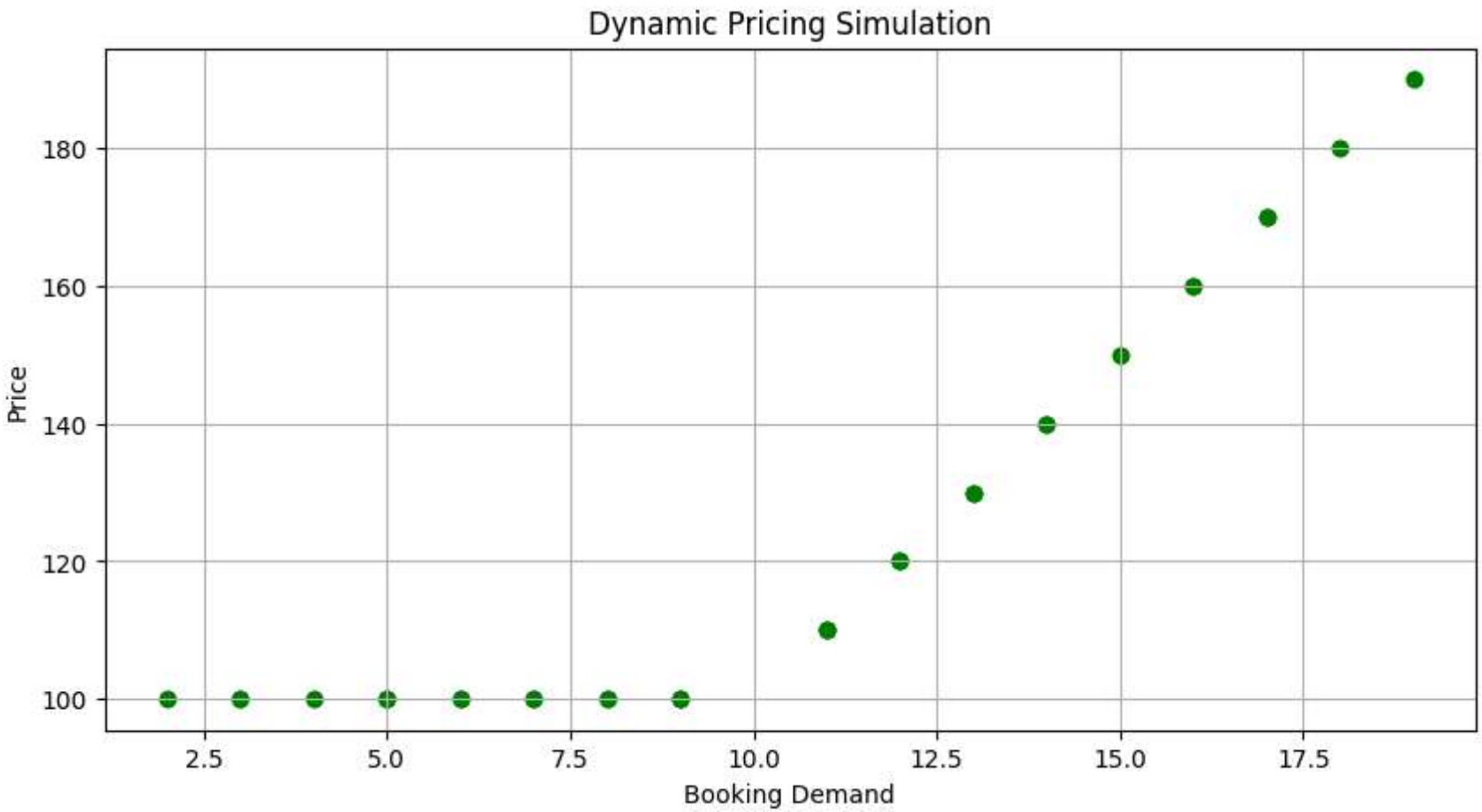
```python
# ----------------------------
# Step 6: Save Results
# ----------------------------
df.to_excel("logistics_pricing_routes.xlsx", index=False)
print("Results saved to 'logistics_pricing_routes.xlsx'")
```

```
Dataset Loaded:
```

	Delivery_ID	City	Distance_km	Booking_Demand	Fuel_Consumption_L	Delivery_Date
0	1	Agra	73	15	8	2026-03-01
1	2	Delhi	32	11	5	2026-03-02
2	3	Jaipur	18	5	12	2026-03-03
3	4	Noida	64	17	10	2026-03-04
4	5	Gurgaon	55	8	4	2026-03-05

	Delivery_ID	City	Distance_km	Booking_Demand	Fuel_Consumption_L	Delivery_Date	Base_Price	Price
0	1	Agra	73	15	8	2026-03-01	100	150
1	2	Delhi	32	11	5	2026-03-02	100	110
2	3	Jaipur	18	5	12	2026-03-03	100	100
3	4	Noida	64	17	10	2026-03-04	100	170
4	5	Gurgaon	55	8	4	2026-03-05	100	100

```
Optimized Route: ['Agra', 'Gurgaon', 'Jaipur', 'Delhi', 'Noida', 'Agra']
```




```
Results saved to 'logistics_pricing_routes.xlsx'
```

In [2]:

```python
# ----------------------------
# Project 2: Logistics Pricing & Route Optimization
# Full Python + SQL + OR-Tools workflow
# ----------------------------

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from ortools.constraint_solver import pywrapcp, routing_enums_pb2
import sqlite3

# ----------------------------
```

```python
# Step 1: Load Excel Data
# ----------------------------
df = pd.read_excel("logistics_data.xlsx")
print("Dataset Loaded:")
display(df.head())

# ----------------------------
# Step 2: Basic Cleaning
# ----------------------------
df.fillna(0, inplace=True)  # Fill missing values

# ----------------------------
# Step 3: Dynamic Pricing Simulation
# ----------------------------
df['Base_Price'] = 100
df['Price'] = df['Booking_Demand'].apply(lambda x: 120 if x>10 else 100)
display(df.head())

# ----------------------------
# Step 4: SQLite Setup and SQL Queries
# ----------------------------
# Connect / create SQLite database
conn = sqlite3.connect("logistics.db")

# Load dataframe into SQL table
df.to_sql('logistics', conn, index=False, if_exists='replace')
print("Data loaded into SQLite table 'logistics'.")

# SQL Query 1: High-demand deliveries
high_demand_df = pd.read_sql("SELECT * FROM logistics WHERE Booking_Demand > 10", conn)
print("High-demand deliveries:")
display(high_demand_df.head())

# SQL Query 2: Total fuel consumption per city
fuel_df = pd.read_sql("""
SELECT City, SUM(Fuel_Consumption_L) AS Total_Fuel
FROM logistics
GROUP BY City
ORDER BY Total_Fuel DESC
""", conn)
print("Fuel consumption per city:")
display(fuel_df)
```

```python
# SQL Query 3: Average distance per city
dist_df = pd.read_sql("""
SELECT City, AVG(Distance_km) AS Avg_Distance
FROM logistics
GROUP BY City
ORDER BY Avg_Distance DESC
""", conn)
print("Average distance per city:")
display(dist_df)

# ----------------------------
# Step 5: Route Optimization (using OR-Tools)
# ----------------------------
cities = high_demand_df['City'].unique()
num_cities = len(cities)

# Random distance matrix (toy example)
np.random.seed(42)
distance_matrix = np.random.randint(5,50,(num_cities,num_cities))
for i in range(num_cities):
    distance_matrix[i,i] = 0

# OR-Tools setup
manager = pywrapcp.RoutingIndexManager(num_cities, 1, 0)
routing = pywrapcp.RoutingModel(manager)

def distance_callback(from_index, to_index):
    return distance_matrix[manager.IndexToNode(from_index)][manager.IndexToNode(to_index)]

transit_callback_index = routing.RegisterTransitCallback(distance_callback)
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)

search_params = pywrapcp.DefaultRoutingSearchParameters()
search_params.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
solution = routing.SolveWithParameters(search_params)

# Extract optimized route
route = []
if solution:
    index = routing.Start(0)
    while not routing.IsEnd(index):
```

```
        route.append(cities[manager.IndexToNode(index)])
        index = solution.Value(routing.NextVar(index))
    route.append(cities[manager.IndexToNode(index)])

print("Optimized Route:", route)

# ----------------------------
# Step 6: Visualization
# ----------------------------
plt.figure(figsize=(10,5))
plt.scatter(df['Booking_Demand'], df['Price'], color='green')
plt.xlabel("Booking Demand")
plt.ylabel("Price")
plt.title("Dynamic Pricing Simulation")
plt.grid(True)
plt.show()

# Optional: plot city-wise total fuel
plt.figure(figsize=(10,5))
plt.bar(fuel_df['City'], fuel_df['Total_Fuel'], color='orange')
plt.xlabel("City")
plt.ylabel("Total Fuel Consumption (L)")
plt.title("Fuel Consumption per City")
plt.show()

# ----------------------------
# Step 7: Save Results
# ----------------------------
df.to_excel("logistics_pricing_routes.xlsx", index=False)
fuel_df.to_excel("fuel_per_city.xlsx", index=False)
dist_df.to_excel("avg_distance_per_city.xlsx", index=False)
print("All results saved to Excel.")

# ----------------------------
# Step 8: Close SQL Connection
# ----------------------------
conn.close()
```

```
Dataset Loaded:
```

	Delivery_ID	City	Distance_km	Booking_Demand	Fuel_Consumption_L	Delivery_Date
0	1	Agra	73	15	8	2026-03-01
1	2	Delhi	32	11	5	2026-03-02
2	3	Jaipur	18	5	12	2026-03-03
3	4	Noida	64	17	10	2026-03-04
4	5	Gurgaon	55	8	4	2026-03-05

	Delivery_ID	City	Distance_km	Booking_Demand	Fuel_Consumption_L	Delivery_Date	Base_Price	Price
0	1	Agra	73	15	8	2026-03-01	100	120
1	2	Delhi	32	11	5	2026-03-02	100	120
2	3	Jaipur	18	5	12	2026-03-03	100	100
3	4	Noida	64	17	10	2026-03-04	100	120
4	5	Gurgaon	55	8	4	2026-03-05	100	100

```
Data loaded into SQLite table 'logistics'.
High-demand deliveries:
```

	Delivery_ID	City	Distance_km	Booking_Demand	Fuel_Consumption_L	Delivery_Date	Base_Price	Price
0	1	Agra	73	15	8	2026-03-01 00:00:00	100	120
1	2	Delhi	32	11	5	2026-03-02 00:00:00	100	120
2	4	Noida	64	17	10	2026-03-04 00:00:00	100	120
3	6	Agra	41	12	7	2026-03-06 00:00:00	100	120
4	8	Delhi	61	14	13	2026-03-08 00:00:00	100	120

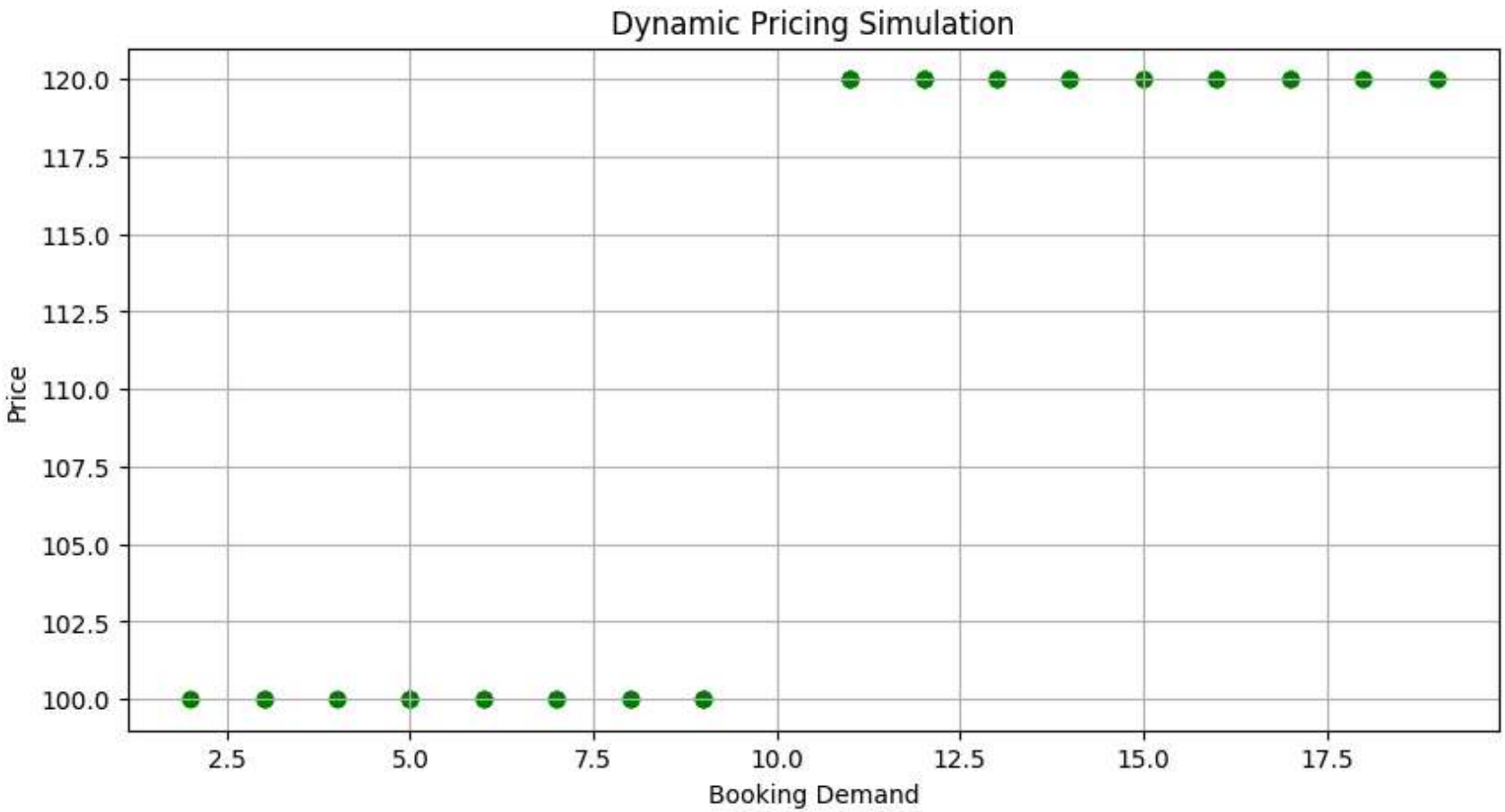
```
Fuel consumption per city:
```

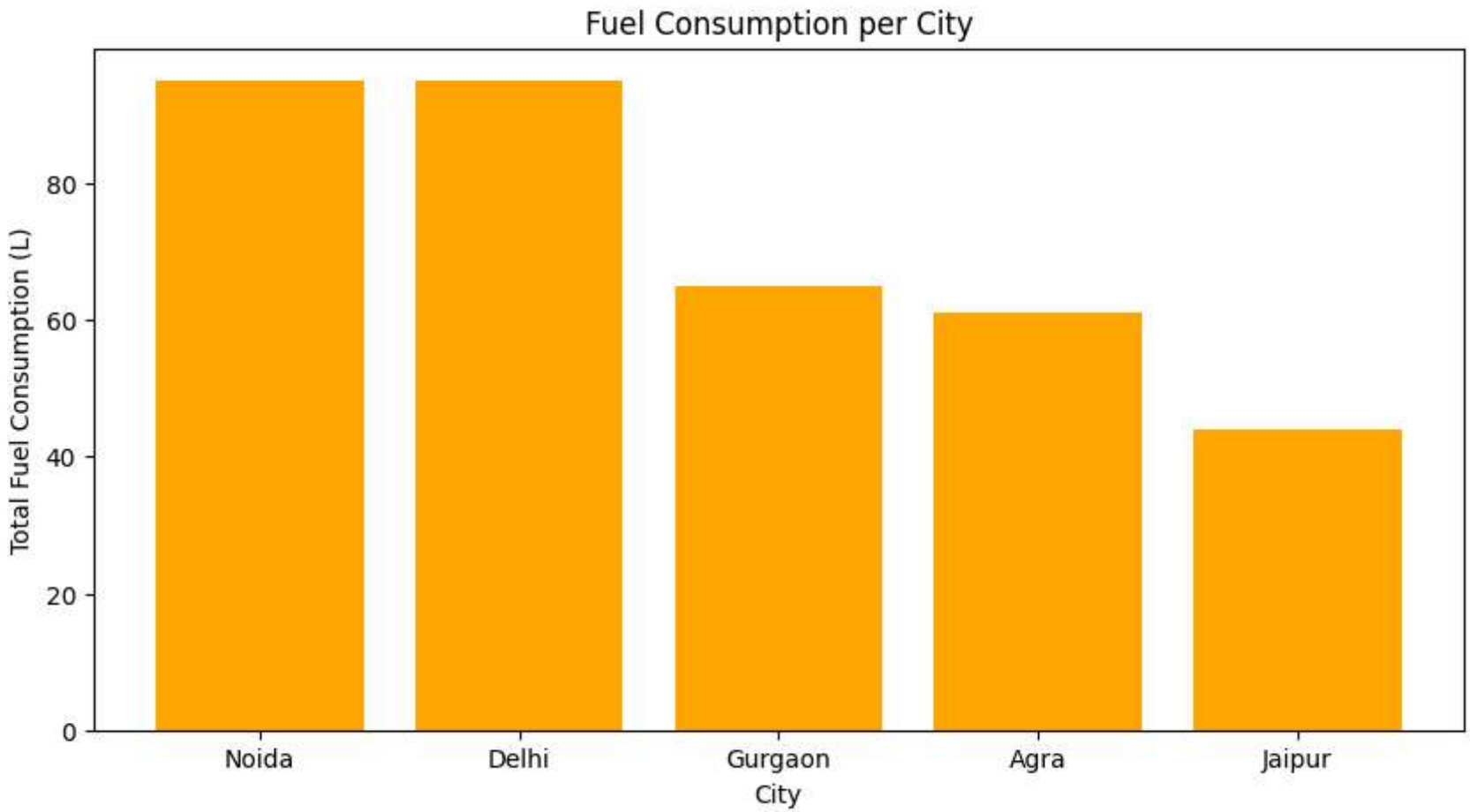
	City	Total_Fuel
0	Noida	95
1	Delhi	95
2	Gurgaon	65
3	Agra	61
4	Jaipur	44

```
Average distance per city:
```

	City	Avg_Distance
0	Noida	69.5
1	Delhi	56.2
2	Gurgaon	48.6
3	Agra	39.7
4	Jaipur	21.4

```
Optimized Route: ['Agra', 'Noida', 'Delhi', 'Gurgaon', 'Agra']
```


Dynamic Pricing Simulation
Price
120.0
117.5
115.0
112.5
110.0
107.5
105.0
102.5
100.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Booking Demand




```
All results saved to Excel.
```

In [ ]: